

Miway CreditCore — Loan Management System Constitution

This document defines the five bounded concepts that make up the lending domain, their fields, relationships, state machines, and the invariant that ties them together. It is the backbone every future schema or feature decision should be checked against.

Why five concepts, not two

The system used to conflate five distinct financial ideas into two database tables: a loans table that was simultaneously “what a client asked for” and “what was actually granted,” and a payments table that was simultaneously “the repayment plan” and “money that actually arrived” — with no link between a payment and the installment it satisfied, and no ledger recording how a balance arrived at its current value. That conflation made it structurally impossible to answer ordinary questions a lender needs to answer: *what did the client ask for, versus what did we grant? Which installment does this payment cover? Why is the balance what it is, entry by entry?*

The correction: separate the request from the grant, the plan from the money, and add a ledger that is the actual source of truth for what’s owed.

The Five Entities

1. LoanApplication — the request and the decision

Holds nothing about money actually moving. Represents a client’s request and whatever decision was made on it.

Field	Meaning
client_id	Who is asking
requested_amount, requested_term_months, purpose	What they asked for
risk_level, risk_score	FraudDetector’s bucket and raw score at decision time — kept even after a decision, so risk judgments stay auditable
status	pending – approved rejected withdrawn (all terminal)
decided_at, decided_by_id	When and who decided
rejection_reason	Required whenever status becomes rejected

State machine: pending is the only state that transitions anywhere. Once approved, rejected, or withdrawn, an application never changes state again. approved is the one transition with a side effect: it creates exactly one LoanAccount.

2. LoanAccount — the credit actually extended

Created only when an application is approved. One-to-one with its application (a unique index enforces this — one decision produces exactly one account; there is no concept of re-approving into a second account).

Field	Meaning
<code>loan_application_id</code>	The decision that created this account (unique)
<code>client_id</code>	Denormalized from the application — every balance/dashboard query filters by client, and it's immutable for the account's life, so the small duplication buys real query simplicity
<code>principal_amount</code> , <code>interest_rate</code> , <code>term_months</code>	What was actually granted, locked in at approval
<code>opened_at</code>	When the account was created
<code>status</code>	active → closed (balance reached zero) written_off defaulted
<code>outstanding_balance</code>	A materialized cache — see the Ledger Invariant below
<code>closed_at</code>	When the account left active

Important correction made here: `outstanding_balance` is seeded from the *total scheduled repayment* (principal + interest, from `InterestCalculator`), not the principal alone. The old system seeded its balance from principal only but then decremented it by full payments (principal + interest) — a figure that could never reconcile against its own repayment schedule. The account's opening balance and the schedule's total are now the same number by construction.

3. RepaymentScheduleInstallment — the plan

One row per scheduled payment, generated once at approval. Never touched by money-received logic directly.

Field	Meaning
<code>loan_account_id</code> , <code>installment_number</code> , <code>due_date</code>	Which account, which installment, when it's due
<code>scheduled_amount</code> , <code>scheduled_principal</code> , <code>scheduled_interest</code>	What's due, split by the amortization schedule
<code>paid_amount</code>	Cumulative amount allocated against this row so far
<code>status</code>	upcoming → overdue (time-based, via the

Field	Meaning
	arrears sweep) or <code>— partially_paid / paid</code> (money-based, via payment allocation)
<code>paid_at</code>	Stamped once <code>paid_amount</code> reaches <code>scheduled_amount</code>

4. PaymentTransaction + PaymentAllocation — money actually received

A `PaymentTransaction` is a fact: money arrived. `amount`, `received_at`, `method`, `reference`, `recorded_by_id`, `notes`, and a status of posted or voided.

Never hard-deleted. Correcting a mis-entered payment means voiding it: status flips to voided, its allocations are reversed, and a compensating reversal ledger entry is posted. The audit trail never loses a row.

Allocation is a join table, not a direct FK, because a single transaction can satisfy more than one installment (a client catching up after being late) and a single installment can be satisfied across more than one transaction (partial payments) — both ordinary in loan servicing. `PaymentAllocation` records `payment_transaction_id`, `repayment_schedule_installment_id`, and `allocated_amount`.

Allocation algorithm: oldest-due-date-first. An incoming payment is applied to the account's unpaid installments in `due_date` order, filling each one before moving to the next, until the amount is exhausted.

Overpayment policy: rejected. An amount exceeding `outstanding_balance` is a changeset error, not a credit balance — prepayment/credit handling is a deliberately deferred feature, not something half-built into this pass.

5. AccountingEntry — the ledger

An immutable, insert-only row per financial event on an account. This is the single source of truth for what an account owes.

Field	Meaning
<code>loan_account_id</code>	Which account
<code>entry_type</code>	<code>disbursement</code> <code>repayment</code> <code>fee</code> <code>write_off</code> <code>reversal</code>
<code>amount</code>	Signed. Positive increases what's owed (disbursement), negative decreases it (repayment)
<code>running_balance</code>	The balance immediately after this entry
<code>source_type</code> , <code>source_id</code>	What caused this entry — a <code>PaymentTransaction</code> , the account itself (disbursement), or a manual adjustment

Field	Meaning
<code>recorded_by_id</code>	Who caused it — null for system-generated entries
<code>occurred_at</code>	The real-world time of the underlying event

This is a **signed-amount, per-account subledger** — one account’s balance history — rather than a full double-entry general ledger with debit/credit columns across a chart of accounts. That’s the right amount of structure for what this system actually tracks today; `entry_type` leaves a clear path to a real GL later without a redesign. No update function is exposed anywhere in the codebase for this table — the same immutability discipline `AuditLogs` already uses.

The Ledger Invariant

The ledger is truth. `LoanAccount.outstanding_balance` is a cache.

Every operation that changes what an account owes — approval (disbursement), a payment (repayment), a write-off — inserts an `AccountingEntry` **and** updates `outstanding_balance` inside the same database transaction (`Ecto.Multi`). Because both writes are atomic, the cache cannot observably drift from the ledger under normal operation.

`MiwayCreditCore.Loans.rebuild_outstanding_balance/1` sums every ledger entry for an account and returns what the balance *should* be. It exists as: - An admin recovery tool, if the cache is ever suspected to have drifted. - A test invariant — every test that touches money asserts `rebuild_outstanding_balance/1` still equals the cached value.

If those two numbers ever disagree in production, that is a bug to investigate immediately, not a state to design around.

Context Boundaries

One public-facing module, `MiwayCreditCore.Loans`, is the single import site every caller uses (`alias MiwayCreditCore.Loans`). Internally it delegates to five submodules, each owning one concept:

Submodule	Owns
<code>Loans.Applications</code>	Submission, the fraud/cooldown guards, approval (which also creates the account, schedule, and disbursement entry), rejection
<code>Loans.Servicing</code>	Account lifecycle — closing, writing off. Named <code>Servicing</code> , not <code>Accounts</code> , to avoid colliding with the unrelated auth <code>MiwayCreditCore.Accounts</code> context
<code>Loans.Schedule</code>	Installment queries and the arrears sweep (<code>mark_overdue_installments/0</code>)
<code>Loans.Payments</code>	Recording and voiding payments —

Submodule	Owns
Loans . Ledger	allocation, ledger entries, balance updates Reading the ledger and rebuilding/verifying the balance

Collateral CRUD stays directly on the Loans facade — small and unchanged in shape, just repointed at LoanAccount instead of the old Loan.

What This Rules Out Going Forward

- No field or table should ever again let “what was requested” and “what was granted” share a row.
- No table should ever again let “the plan” and “money received” share a row without an explicit link between them.
- No balance field should ever again be mutated without a corresponding ledger entry in the same transaction.
- Collateral, and anything else that secures a loan, attaches to a LoanAccount — never to a request that hasn’t been decided yet.